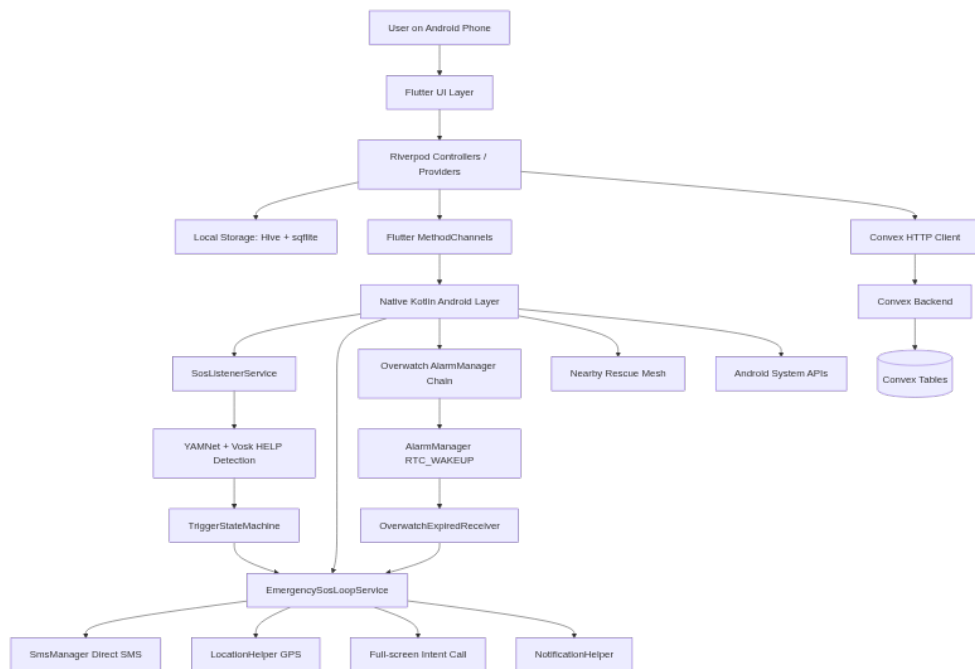


# ProteqMe

Android Safety Companion with SOS Automation, Voice Detection, Dead-Man's Switch Timer, and Offline Rescue Mesh



Detailed Project Report and Usage Study

Prepared for Overleaf PDF Compilation

May 17, 2026

# Contents

|                                               |           |
|-----------------------------------------------|-----------|
| <b>Executive Summary</b>                      | <b>1</b>  |
| <b>1 Introduction</b>                         | <b>2</b>  |
| 1.1 Background . . . . .                      | 2         |
| 1.2 Project Goal . . . . .                    | 2         |
| 1.3 Target Beneficiaries . . . . .            | 2         |
| <b>2 System Overview</b>                      | <b>4</b>  |
| 2.1 High-Level Architecture . . . . .         | 4         |
| 2.2 Design Principle . . . . .                | 5         |
| <b>3 User Experience and Screens</b>          | <b>6</b>  |
| 3.1 User Flow . . . . .                       | 7         |
| 3.2 Home Screen . . . . .                     | 8         |
| 3.3 Important Screens . . . . .               | 8         |
| <b>4 Emergency Workflow</b>                   | <b>9</b>  |
| 4.1 Emergency Sequence . . . . .              | 9         |
| 4.2 Why SMS Comes Before Call . . . . .       | 10        |
| 4.3 Biometric Disarm . . . . .                | 10        |
| <b>5 Voice Detection</b>                      | <b>11</b> |
| 5.1 Voice Detection Pipeline . . . . .        | 11        |
| 5.2 Trigger Protection . . . . .              | 11        |
| <b>6 Overwatch / Safe Journey Timer</b>       | <b>12</b> |
| 6.1 Purpose . . . . .                         | 12        |
| 6.2 Overwatch Flow . . . . .                  | 13        |
| 6.3 States . . . . .                          | 14        |
| 6.4 Why Native AlarmManager Is Used . . . . . | 14        |
| 6.5 Reboot Handling . . . . .                 | 14        |
| <b>7 Contacts, Profile, and Local Data</b>    | <b>15</b> |
| 7.1 Contact and Profile Data Flow . . . . .   | 15        |
| 7.2 Emergency Contacts . . . . .              | 15        |

|           |                                         |           |
|-----------|-----------------------------------------|-----------|
| 7.3       | User Display Name . . . . .             | 16        |
| 7.4       | Offline Reliability . . . . .           | 16        |
| <b>8</b>  | <b>Rescue Mode</b>                      | <b>17</b> |
| 8.1       | Rescue Architecture . . . . .           | 17        |
| 8.2       | Victim Role . . . . .                   | 17        |
| 8.3       | Rescuer Role . . . . .                  | 17        |
| 8.4       | Disaster Usage . . . . .                | 17        |
| <b>9</b>  | <b>Convex Backend</b>                   | <b>18</b> |
| 9.1       | Convex Data Model . . . . .             | 18        |
| 9.2       | Cloud Responsibilities . . . . .        | 18        |
| 9.3       | Why Cloud Is Optional . . . . .         | 19        |
| <b>10</b> | <b>Native Android Components</b>        | <b>20</b> |
| 10.1      | Native Component Map . . . . .          | 20        |
| 10.2      | Key Native Components . . . . .         | 20        |
| <b>11</b> | <b>Use Case Analysis</b>                | <b>22</b> |
| 11.1      | Miners and Industrial Workers . . . . . | 22        |
| 11.2      | Disaster Victims . . . . .              | 22        |
| 11.3      | Child and Parent Safety . . . . .       | 22        |
| 11.4      | Normal People . . . . .                 | 23        |
| 11.5      | Elderly Users and Patients . . . . .    | 23        |
| <b>12</b> | <b>Permissions and Device Setup</b>     | <b>24</b> |
| 12.1      | Android Permissions . . . . .           | 24        |
| 12.2      | Poco, Xiaomi, and HyperOS . . . . .     | 24        |
| <b>13</b> | <b>Security, Privacy, and Ethics</b>    | <b>25</b> |
| 13.1      | Security Controls . . . . .             | 25        |
| 13.2      | Privacy Considerations . . . . .        | 25        |
| 13.3      | Child Safety Ethics . . . . .           | 25        |
| <b>14</b> | <b>Limitations and Future Work</b>      | <b>26</b> |
| 14.1      | Current Limitations . . . . .           | 26        |
| 14.2      | Future Work . . . . .                   | 26        |
| <b>15</b> | <b>Conclusion</b>                       | <b>27</b> |
| <b>A</b>  | <b>Image Assets Used in This Report</b> | <b>28</b> |
| <b>B</b>  | <b>Overleaf Compilation Notes</b>       | <b>29</b> |

# List of Figures

|      |                                                            |    |
|------|------------------------------------------------------------|----|
| 2.1  | High-level ProteqMe architecture . . . . .                 | 4  |
| 3.1  | Main user flow from app open to emergency disarm . . . . . | 7  |
| 3.2  | Home screen layout hierarchy . . . . .                     | 8  |
| 4.1  | Emergency workflow sequence . . . . .                      | 9  |
| 5.1  | Voice detection pipeline . . . . .                         | 11 |
| 6.1  | Overwatch or Safe Journey timer flow . . . . .             | 13 |
| 7.1  | Contact and profile data flow . . . . .                    | 15 |
| 8.1  | Victim and rescuer rescue mode . . . . .                   | 17 |
| 9.1  | Convex backend entity relationship diagram . . . . .       | 18 |
| 10.1 | Native Android component relationship . . . . .            | 20 |

# Executive Summary

ProteqMe is an Android-first personal safety application designed to help people during high-risk situations such as harassment, accidents, isolated travel, disaster displacement, underground work, child safety monitoring, and emergency rescue coordination. The project combines Flutter for user experience, native Kotlin for emergency execution, local offline storage for reliability, and Convex for optional cloud backup and synchronization.

The central purpose of the system is simple: when the user is in danger, the app should quickly alert trusted contacts with useful context. This includes SMS alerts, GPS location, timestamp, the user's name, an automatic call to the primary contact, and an in-app emergency overlay with biometric disarm. The project also includes SOS listening mode, an Overwatch or Safe Journey timer, offline rescue mode, contact import, HyperOS/Poco setup guidance, and Convex cloud support.

This report explains the complete system design, the target users, the usage scenarios, the technical architecture, the emergency workflow, the native Android services, the Convex backend, and the ethical considerations needed for safe deployment.

# 1 Introduction

## 1.1 Background

Many emergency apps fail at the moment they are needed most because they depend only on a foreground screen, internet access, or manual user interaction. In real emergencies, the user may not be able to unlock the phone, open the app, type a message, or explain their location. Android devices also apply strict background restrictions, especially on devices running vendor ROMs such as Xiaomi HyperOS, MIUI, and other aggressive battery managers.

ProteqMe addresses these problems by moving safety-critical work into native Android components. Flutter remains responsible for the user interface and day-to-day interaction, while Kotlin foreground services, broadcast receivers, AlarmManager, SmsManager, location APIs, and full-screen intent notifications handle emergency execution.

## 1.2 Project Goal

The goal of ProteqMe is to provide a practical Android safety companion that can:

- Trigger emergency alerts through a visible SOS button.
- Detect emergency voice cues through background listening.
- Send SMS alerts to all emergency contacts before starting a call.
- Include user name, timestamp, location link, and trigger context in messages.
- Require biometric or device PIN authentication to cancel emergency states.
- Support a Safe Journey or Dead-Man's Switch timer for preventative safety.
- Provide offline rescue discovery using nearby device communication.
- Store emergency-critical data locally so it works without internet.
- Optionally sync profile, contacts, live location, and SOS events to Convex.

## 1.3 Target Beneficiaries

ProteqMe is useful for multiple user groups:

- **Miners and industrial workers:** Workers in isolated or underground environments can use SOS listening, Overwatch timers, and emergency contacts when accidents happen.
- **Disaster victims:** During floods, landslides, earthquakes, storms, and evacuation events, the rescue mesh and location messages can help responders locate victims.
- **Children and parent safety:** Parents can configure trusted emergency contacts and use Safe Journey timers for school travel, tuition travel, or late travel.
- **Normal people:** Anyone walking alone, traveling at night, using ride services, visiting unfamiliar places, or facing medical risk can use the app.
- **Elderly users and patients:** People with health risks can use the SOS button or voice trigger to alert caretakers.
- **Rescue volunteers:** Volunteers can use rescuer mode to discover nearby victims who are broadcasting an SOS signal.

## 2 System Overview

### 2.1 High-Level Architecture

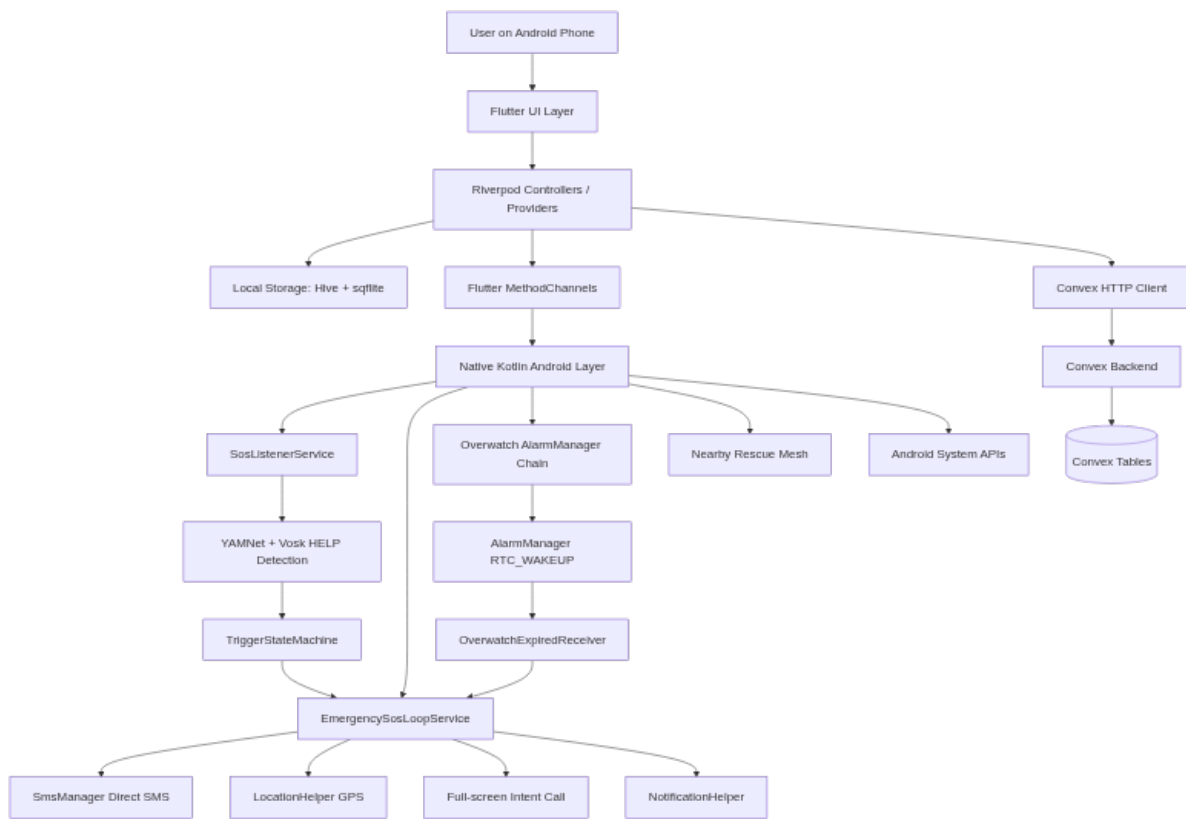


Figure 2.1: High-level ProteqMe architecture

The system is organized into four major layers:

1. **Flutter application layer:** Screens, navigation, Riverpod state, forms, permissions UI, contacts UI, settings, rescue UI, profile, logs, and emergency overlay.
2. **Local storage layer:** Hive and sqflite store contacts, SOS state, user display name, logs, and emergency-critical data.
3. **Native Android layer:** Kotlin services and receivers handle voice detection, emergency execution, SMS, calls, GPS, Overwatch alarms, reboot recovery, and notifications.



4. **Cloud companion layer:** Convex provides optional cloud authentication, contact sync, live location, and SOS event storage.

## 2.2 Design Principle

The most important design principle is that emergency behavior must not depend only on Flutter foreground execution. Flutter timers can stop when the app is killed. Background tasks can be restricted. Internet can disappear. For that reason, the emergency execution path is delegated to native Android components wherever possible.

## 3 User Experience and Screens

### 3.1 User Flow

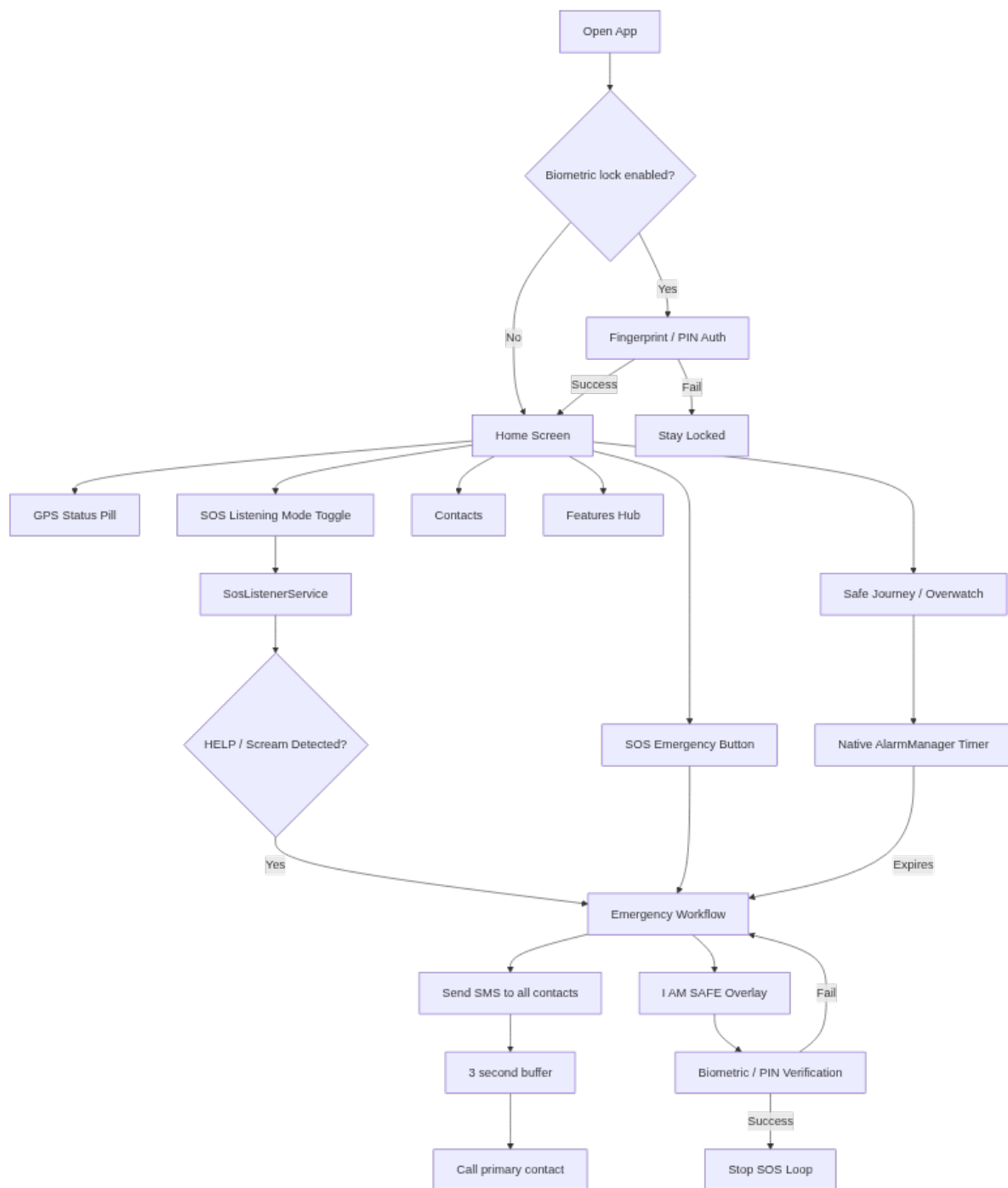


Figure 3.1: Main user flow from app open to emergency disarm

### 3.2 Home Screen

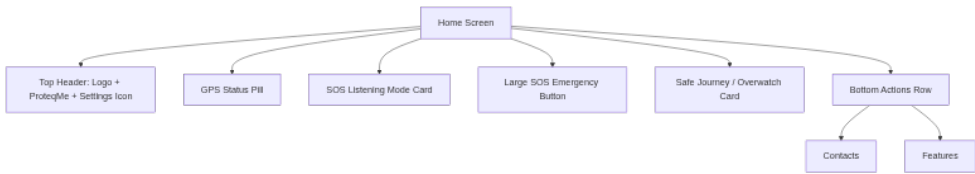


Figure 3.2: Home screen layout hierarchy

The home screen is organized for fast emergency access. The top of the screen shows the app name, logo, and settings icon. The next section shows the GPS status pill, followed by the SOS listening card. The large SOS emergency button is the visual center of the screen. Below the emergency area, the user can access contacts and features.

The manual trigger label was removed to simplify the main emergency interaction. The SOS button is now the main visible emergency trigger.

### 3.3 Important Screens

| Screen            | Purpose                                                                               |
|-------------------|---------------------------------------------------------------------------------------|
| Home              | Shows GPS status, listening mode, SOS button, Overwatch card, contacts, and features. |
| Contacts          | Add, edit, delete, import, and prioritize emergency contacts.                         |
| Features Hub      | Access profile, rescue mode, logs, permissions, device setup, and other tools.        |
| Profile           | Stores the user’s display name used in outgoing SOS messages.                         |
| Rescuer Mode      | Allows rescuers to scan for victim devices broadcasting SOS rescue signals.           |
| Permissions       | Shows required Android permissions and prompts the user to grant them.                |
| Device Setup      | Provides Poco, Xiaomi, HyperOS, and battery optimization instructions.                |
| Emergency Overlay | Shows the active SOS state and the biometric-gated I AM SAFE button.                  |
| Auth              | Connects the user with Convex cloud identity and sync features.                       |

## 4 Emergency Workflow

### 4.1 Emergency Sequence

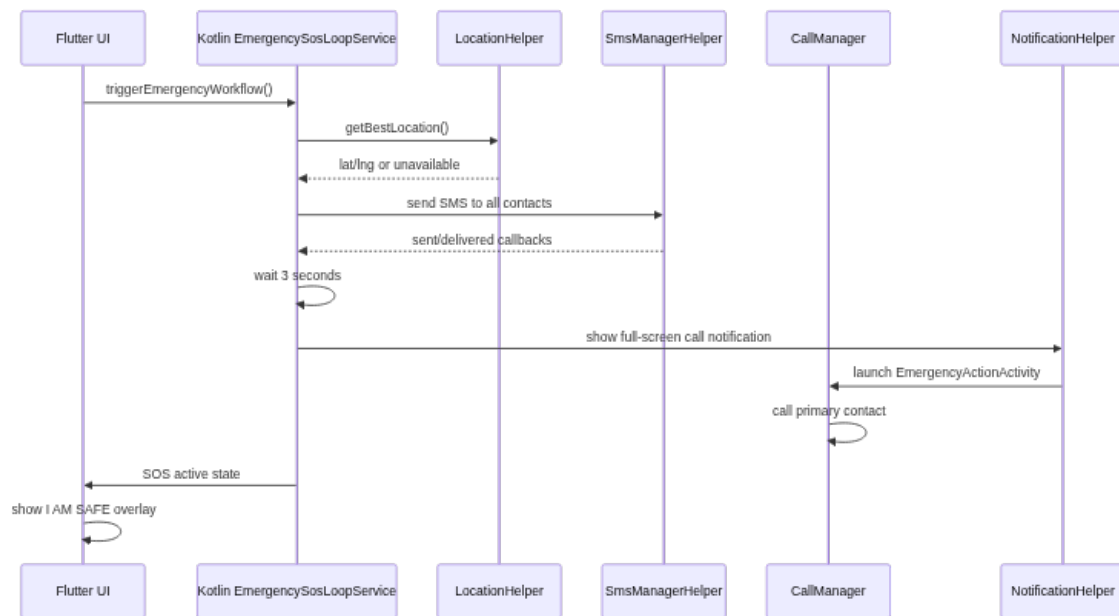


Figure 4.1: Emergency workflow sequence

The emergency workflow follows a strict order:

1. Validate emergency contacts.
2. Fetch the best available location.
3. Build an emergency SMS message with name, timestamp, location, and trigger context.
4. Send SMS to all emergency contacts.
5. Wait briefly so SMS has time to dispatch.
6. Trigger a full-screen intent call to the primary contact.
7. Keep SOS active until the user verifies safety through biometric or device PIN.

## 4.2 Why SMS Comes Before Call

SMS is sent before calling because it carries structured emergency information. A phone call may fail, be missed, or be disconnected. SMS can still provide the contact with the user's name, approximate location, timestamp, and reason for alert. After SMS dispatch, the app starts the call flow.

## 4.3 Biometric Disarm

The I AM SAFE button is protected using Android biometric or device credential authentication. This prevents accidental or forced cancellation. If authentication fails, the emergency workflow continues.

# 5 Voice Detection

## 5.1 Voice Detection Pipeline

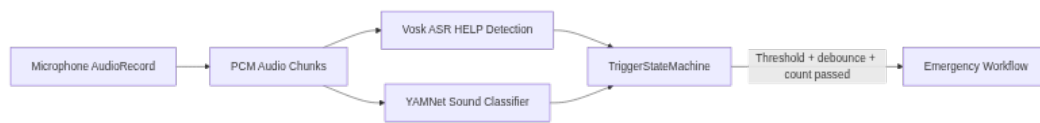


Figure 5.1: Voice detection pipeline

SOS listening mode uses the microphone through a native Android foreground service. Audio is processed through a voice and sound detection pipeline. The project uses HELP recognition and sound classification ideas through components such as Vosk ASR and YAMNet-style processing.

## 5.2 Trigger Protection

The detection logic should avoid accidental emergency activation. For this reason, the system uses:

- Trigger windows.
- Detection count thresholds.
- Debounce periods.
- Cooldown intervals.
- Configurable trigger modes.

These controls reduce false positives while still allowing emergency detection when the user cannot manually press the SOS button.

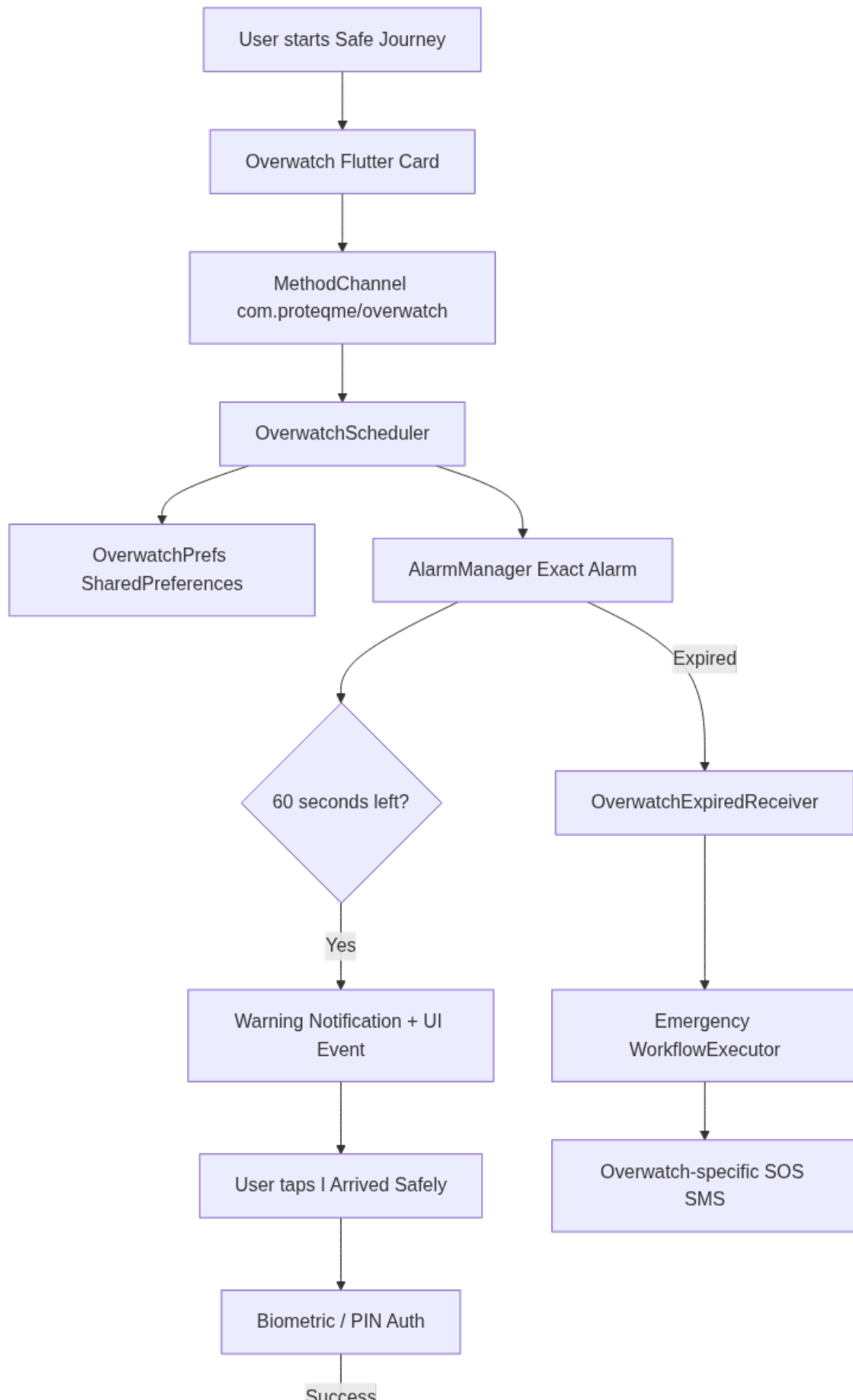
## 6 Overwatch / Safe Journey Timer

### 6.1 Purpose

The Overwatch feature is a Dead-Man's Switch style timer. The user sets a journey duration such as 15 minutes, 30 minutes, 1 hour, or 2 hours. The app starts a native Android countdown. If the user does not confirm arrival before the timer expires, the app automatically triggers the standard SOS workflow.



## 6.2 Overwatch Flow



## 6.3 States

| State     | Description                                                                     |
|-----------|---------------------------------------------------------------------------------|
| Setup     | User selects duration and optionally enters destination text.                   |
| Active    | App shows countdown timer and I Arrived Safely button.                          |
| Warning   | Last 60 seconds trigger a warning notification, haptic feedback, and urgent UI. |
| Expired   | Native receiver triggers the emergency workflow automatically.                  |
| Completed | User passes biometric or PIN verification and cancels the timer.                |

## 6.4 Why Native AlarmManager Is Used

Flutter timers are not reliable if the app is swiped away or background restricted. Android AlarmManager allows the app to schedule a wake-up event at a specific time. This is more suitable for safety-critical countdowns.

## 6.5 Reboot Handling

The timer stores its end timestamp in SharedPreferences. If the phone reboots, BootReceiver can check the remaining time. If the timer has not expired, it re-registers the alarm. If it already expired during reboot, it can trigger the emergency action immediately after boot.

## 7 Contacts, Profile, and Local Data

### 7.1 Contact and Profile Data Flow

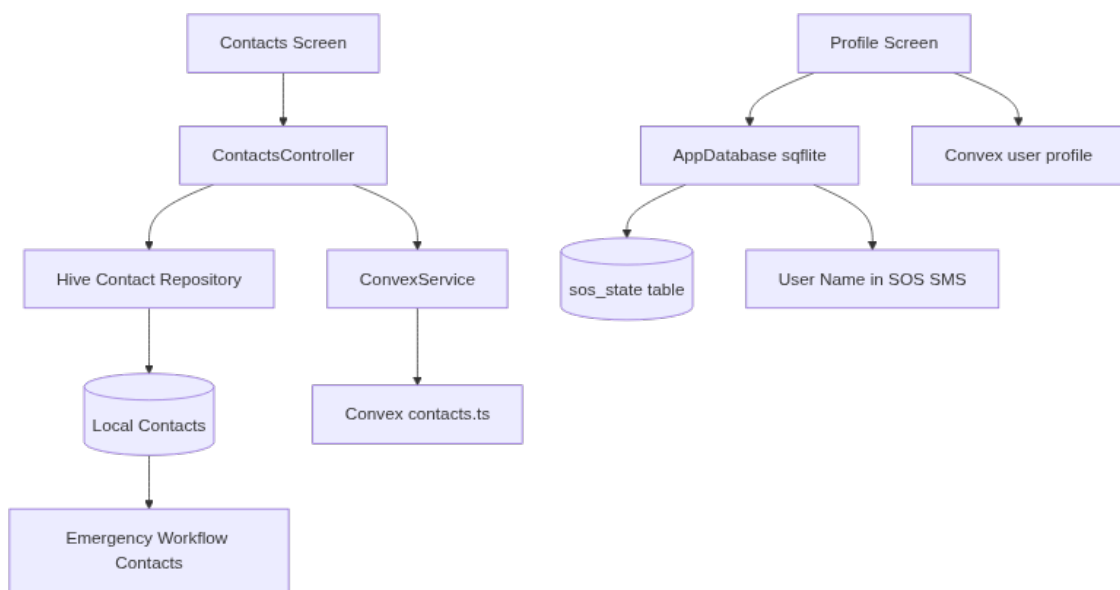


Figure 7.1: Contact and profile data flow

### 7.2 Emergency Contacts

Emergency contacts are core to the app. The user can:

- Add contacts manually.
- Import contacts from the phone contact book.
- Select a primary contact.
- Edit and remove saved contacts.
- Use biometric or device PIN authentication for sensitive contact changes.

## 7.3 User Display Name

The user's display name is used in outgoing SOS messages. This helps contacts understand who is in danger, especially when multiple family members or workers use similar emergency systems.

## 7.4 Offline Reliability

Contacts and display name are stored locally so emergency alerts can work even when there is no internet connection. Convex sync is useful for backup and restoration, but the emergency path should not depend only on the cloud.

# 8 Rescue Mode

## 8.1 Rescue Architecture

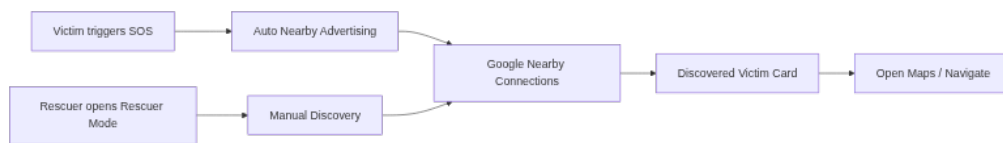


Figure 8.1: Victim and rescuer rescue mode

## 8.2 Victim Role

When SOS is triggered, the victim device can automatically advertise a rescue signal. This signal allows nearby rescuers to discover the victim even when normal internet communication is unavailable or unreliable.

## 8.3 Rescuer Role

The rescuer manually starts rescuer mode. The app scans for nearby victim devices and displays discovered victims. A rescuer can then use available location details to navigate toward the victim.

## 8.4 Disaster Usage

This feature is especially useful during disasters where cellular networks may be overloaded, unavailable, or partially damaged. It can help volunteers identify nearby victims without requiring every victim to manually coordinate through a server.

## 9 Convex Backend

### 9.1 Convex Data Model

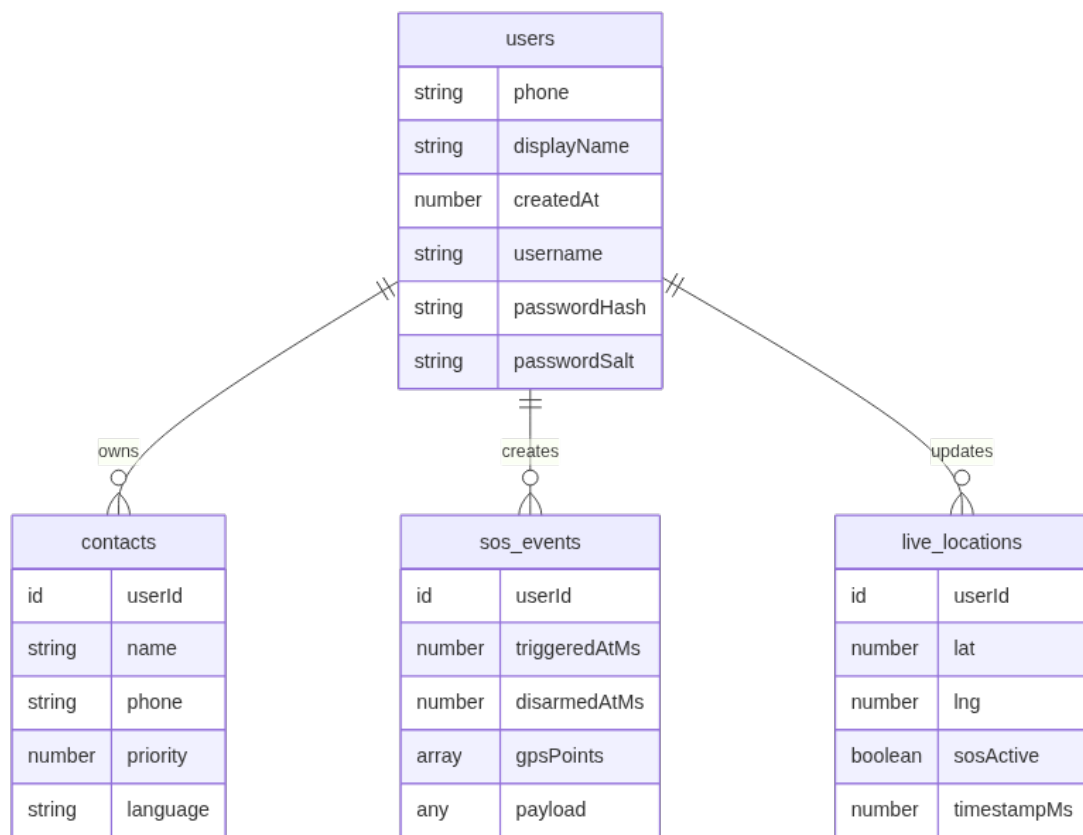


Figure 9.1: Convex backend entity relationship diagram

### 9.2 Cloud Responsibilities

Convex acts as the cloud companion for:

- User identity and profile.
- Contact backup and synchronization.
- Live location mirroring.

- SOS event recording.
- Future cloud escalation workflows.

### 9.3 Why Cloud Is Optional

The app should still work in emergency conditions without internet. Therefore, Convex improves continuity and backup, but emergency execution remains local-first.

# 10 Native Android Components

## 10.1 Native Component Map

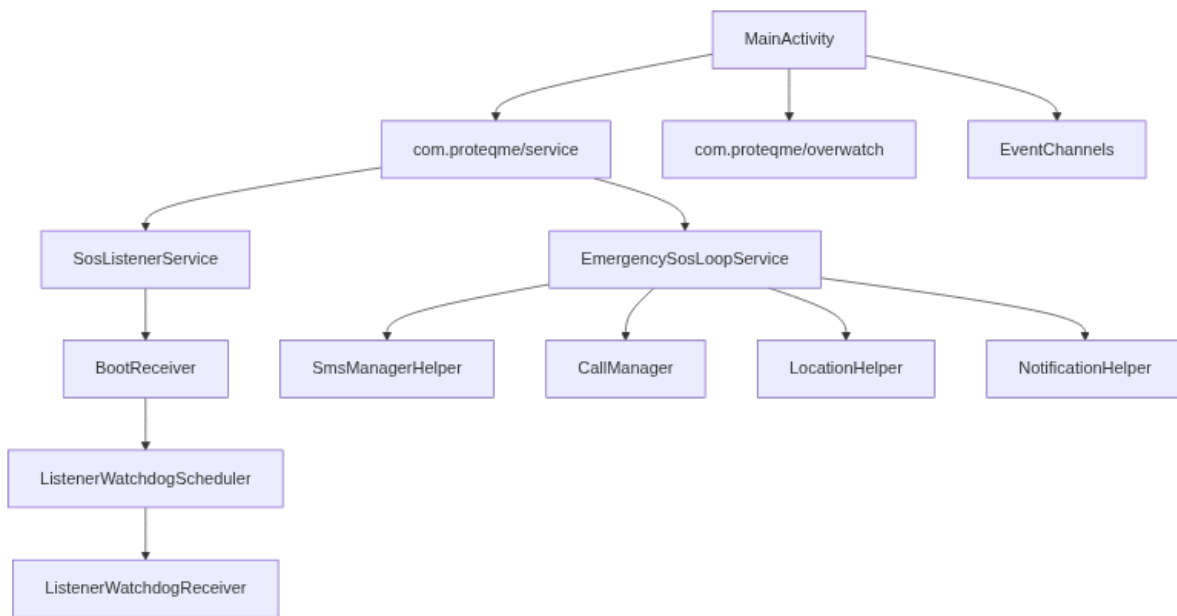


Figure 10.1: Native Android component relationship

## 10.2 Key Native Components

| Component               | Responsibility                                                         |
|-------------------------|------------------------------------------------------------------------|
| MainActivity            | Registers MethodChannels and EventChannels between Flutter and Kotlin. |
| SosListenerService      | Runs background voice detection as a foreground service.               |
| EmergencySosLoopService | Runs the SMS, GPS, call, and SOS loop logic.                           |
| SmsManagerHelper        | Sends direct SMS and tracks sent or delivered status.                  |
| CallManager             | Initiates emergency call behavior through full-screen intent flow.     |



---

|                          |                                                                      |
|--------------------------|----------------------------------------------------------------------|
| LocationHelper           | Fetches best available location for emergency messages.              |
| NotificationHelper       | Shows foreground service, emergency call, and warning notifications. |
| BootReceiver             | Restores listener watchdog and time-sensitive alarms after reboot.   |
| OverwatchScheduler       | Schedules Safe Journey expiry and warning alarms.                    |
| OverwatchExpiredReceiver | Receives Overwatch expiry and starts emergency execution.            |

---

# 11 Use Case Analysis

## 11.1 Miners and Industrial Workers

Mining and industrial work can involve isolation, noise, dust, underground environments, and limited connectivity. ProteqMe can help by providing:

- A fast SOS button for accidents.
- Voice-trigger emergency detection if the user cannot reach the phone.
- Overwatch timers for risky tasks or travel inside work zones.
- SMS alerts to supervisors or emergency contacts.
- Offline rescue signals for nearby responders.

## 11.2 Disaster Victims

During disasters, victims may be trapped, displaced, or unable to communicate normally. ProteqMe can help by:

- Sending SMS with last known location.
- Broadcasting rescue information nearby.
- Allowing rescuers to manually scan for victims.
- Preserving emergency function even when cloud sync is unavailable.

## 11.3 Child and Parent Safety

For children, the app can support safer travel and parent awareness:

- Parents can configure emergency contacts.
- Safe Journey timers can be used during travel to school, tuition, or public transport.
- The app can require fingerprint or device PIN for sensitive actions.
- Contact editing can be protected to avoid accidental or unauthorized changes.

This use case must be handled ethically. The app should be used for safety and consent-based protection, not secret surveillance.

## 11.4 Normal People

Normal users can benefit during:

- Night travel.
- Walking alone.
- Ride-share trips.
- Medical risk situations.
- Travel in unfamiliar areas.
- Domestic safety concerns.

## 11.5 Elderly Users and Patients

The app can help elderly people or patients who may need urgent assistance. A simple SOS button, emergency contacts, and optional voice trigger can reduce the time needed to alert caretakers.

# 12 Permissions and Device Setup

## 12.1 Android Permissions

| Permission                 | Reason                                                     |
|----------------------------|------------------------------------------------------------|
| Microphone                 | Required for SOS listening and voice detection.            |
| Location                   | Required to include GPS map links in SOS messages.         |
| Send SMS                   | Required for direct emergency SMS alerts.                  |
| Call phone                 | Required for emergency calling flow.                       |
| Notifications              | Required for foreground services and warning alerts.       |
| Foreground service         | Required for long-running emergency and listener services. |
| Biometric / fingerprint    | Required for I AM SAFE, contact protection, and app lock.  |
| Bluetooth / nearby devices | Required for rescue mesh advertising and discovery.        |
| Boot completed             | Required to restore watchdogs and timers after re-boot.    |
| Exact alarm                | Required for reliable Overwatch expiry.                    |

## 12.2 Poco, Xiaomi, and HyperOS

Some Android vendors aggressively restrict background apps. For reliable emergency behavior, the user may need to:

- Set battery usage to unrestricted or no restrictions.
- Allow autostart.
- Allow background activity.
- Allow notification display and full-screen notifications.
- Enable SMS sending without confirmation where the device requires it.
- Confirm SIM balance or SMS plan availability.

# 13 Security, Privacy, and Ethics

## 13.1 Security Controls

ProteqMe uses several safety controls:

- Biometric or device PIN for emergency disarm.
- Biometric or device PIN for modifying or removing contacts.
- Optional biometric lock for opening the home screen.
- Local-first emergency data storage.
- Optional cloud sync instead of cloud dependency.

## 13.2 Privacy Considerations

Because the app handles location, contacts, emergency events, and identity, privacy must be treated seriously. Users should know:

- What data is stored locally.
- What data is uploaded to Convex.
- Who receives SOS messages.
- When rescue broadcasting is active.
- How to disable features safely.

## 13.3 Child Safety Ethics

For child-parent safety, the app should be used transparently. Children should understand what the app does, who receives emergency alerts, and how the Safe Journey timer works. The app should not be positioned as a hidden tracking tool.

# 14 Limitations and Future Work

## 14.1 Current Limitations

- The app is Android-only.
- Vendor ROMs may still restrict background execution unless configured.
- SMS delivery depends on SIM credit, carrier behavior, and phone settings.
- If the phone battery dies completely, on-device Overwatch cannot fire.
- Convex cloud functions are optional and require correct environment setup.
- Voice detection can have false positives or false negatives in noisy environments.

## 14.2 Future Work

- Convex and Twilio mirror for zero-battery Overwatch escalation.
- Better cloud retry queue for failed sync.
- Production-grade OTP or username/password authentication.
- More robust offline mesh routing.
- Admin or family dashboard for consent-based safety monitoring.
- Multi-language SOS message templates.
- Wearable integration for fast SOS triggers.

## 15 Conclusion

ProteqMe is a practical Android safety companion designed for real-world emergency constraints. Its strongest architectural decision is separating user interface from emergency execution. Flutter provides a clean and accessible UI, while native Kotlin handles background-safe emergency workflows.

The project can help miners, disaster victims, children and parents, elderly users, travelers, and normal people who need a reliable emergency companion. With continued work on cloud escalation, authentication, and device compatibility, ProteqMe can become a strong safety platform for both personal and community emergency response.

# A Image Assets Used in This Report

This report references the following PNG images. When uploading to Overleaf, keep the same folder name ‘project<sub>d</sub>igrams’nexttothis’.tex’ file.

1. ‘project<sub>d</sub>igrams/01<sub>p</sub>roject<sub>a</sub>rchitecture.png’
1. ‘project<sub>d</sub>igrams/02<sub>u</sub>ser\_flow.png’
1. ‘project<sub>d</sub>igrams/03<sub>h</sub>ome<sub>s</sub>creen<sub>i</sub>ayout.png’
1. ‘project<sub>d</sub>igrams/04<sub>e</sub>mergency<sub>w</sub>ork\_flow.png’
1. ‘project<sub>d</sub>igrams/05<sub>v</sub>oice<sub>d</sub>etection<sub>p</sub>ipeline.png’
1. ‘project<sub>d</sub>igrams/06<sub>o</sub>verwatch<sub>s</sub>a<sub>f</sub>e<sub>j</sub>ourney.png’
1. ‘project<sub>d</sub>igrams/07<sub>c</sub>ontact<sub>p</sub>rofile<sub>d</sub>ata\_flow.png’
1. ‘project<sub>d</sub>igrams/08<sub>r</sub>escue<sub>m</sub>ode.png’
1. ‘project<sub>d</sub>igrams/09<sub>c</sub>onvex<sub>b</sub>ackend.png’
1. ‘project<sub>d</sub>igrams/10<sub>n</sub>ative<sub>a</sub>ndroid<sub>c</sub>omponents.png’



## B Overleaf Compilation Notes

To compile this report in Overleaf:

1. Upload ‘PROTEQME<sub>R</sub>*EPORT.tex*’.
1. Upload the complete ‘project<sub>d</sub>*igrams*’ folder.
1. Set the compiler to ‘pdfLaTeX’.
2. Click Recompile.

If images do not appear, confirm that the folder name is exactly ‘project<sub>d</sub>*igrams*’ and that all PNG files